TaskForge — Smart Task Management System

Detailed Project Report

Date: April 2026 **Technology:** Next.js 16 · TypeScript 5 · MongoDB · React 19

---

1. Project Team

Team Member	Role	Responsibilities
Deepanshu	**Admin**	Full system architecture, backend development, database design, authentication & security, deployment & maintenance
Dev Tyagi	**UI/UX Designer**	User interface design, component styling, animations, responsive layouts, glassmorphism effects, user experience optimization
Kanishk Sharma	**Project Manager**	Project planning, task allocation, team coordination, requirement analysis, workflow optimization
Daniel Tayal	**Developer**	Frontend development, API integration, state management, React components, feature implementation
Ritik Ranjan	**Member**	Testing & quality assurance, documentation, bug reporting, feature feedback, user support

---

2. Introduction

TaskForge is a modern, full-stack task management system designed for team collaboration. The application enables administrators to create projects, manage team members, assign tasks with priorities and deadlines, and monitor progress through a comprehensive dashboard. Team members receive notifications, accept project invitations, and update their task statuses through a dedicated member interface.

The system is built with a strong emphasis on software engineering principles, implementing SOLID design principles, the Service-Repository architectural pattern, and secure JWT-based authentication throughout.

---

3. Project Objectives

- Build a scalable task management platform with role-based access control
- Implement secure authentication using JWT tokens and bcrypt password hashing
- Follow SOLID principles and clean architecture for maintainability
- Create an intuitive admin dashboard for project and team management
- Provide a member interface for task tracking and notifications
- Enforce business rules such as unique project names and task title constraints
- Implement an approval workflow for new member registrations

---

4. Tech Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	16	Server-side rendering, file-based routing, API routes
UI Library	React	19	Component-based UI with hooks and context
Language	TypeScript	5 (strict)	Static typing, interfaces, generics, type safety
Styling	Tailwind CSS	4	Utility-first CSS framework with dark mode

Layer	Technology	Version	Purpose
Animation	Framer Motion	12	Page transitions, scroll animations, micro-interactions
Icons	Lucide React	Latest	Comprehensive SVG icon library
Database	MongoDB + Mongoose	9	NoSQL document database with ODM
Auth (Server)	jsonwebtoken	9	JWT token generation and verification
Auth (Edge)	jose	6	Lightweight JWT verification for edge middleware
Password Security	bcryptjs	3	Salted password hashing (12 rounds)
Date Handling	date-fns	4	Date formatting and deadline calculations

---

5. System Architecture

TaskForge employs a **Layered Architecture** with the **Service-Repository Pattern**, organizing the codebase into five distinct layers with clear separation of concerns:

Architecture Layers

Presentation Layer — React components, pages, and layouts handle all UI rendering, user interaction, and client-side state management. The App Router provides file-based routing. Global state is managed through React Context providers (AuthProvider for authentication, ThemeProvider for theme toggling).

API Layer — Next.js API route handlers serve as the gateway between the frontend and backend. Each route handler validates input, delegates to the appropriate service, and returns structured JSON responses. Routes cover authentication, tasks, projects, users, invitations, and notifications.

Service Layer — Dedicated service classes encapsulate all business logic. Services validate data, enforce business rules (uniqueness constraints, authorization checks), and orchestrate operations that may span multiple repositories. All services extend a BaseService class for consistent error handling and logging.

Repository Layer — Repository classes abstract all database operations, providing a clean interface for data access. Each repository handles CRUD operations and domain-specific queries for a single entity type. All repositories extend a generic BaseRepository class.

Data Layer — Mongoose schemas define the MongoDB document structure, field types, validation constraints, unique indexes, and automatic timestamp generation.

---

6. SOLID Principles Implementation

Single Responsibility Principle (SRP)

Every class in the system has exactly one reason to change:

- **BaseEntity** — Responsible only for ID generation and timestamp management
- **BaseRepository** — Responsible only for generic CRUD database operations
- **BaseService** — Responsible only for cross-cutting concerns (error handling, logging, validation)
- **AuthService** — Handles only authentication: login, signup, JWT generation, session validation
- **UserService** — Handles only user management: CRUD operations, approval workflow, password hashing
- **TaskService** — Handles only task business logic: creation, assignment, uniqueness validation
- **ProjectService** — Handles only project management: creation, member management
- **InvitationService** — Handles only the invitation workflow: create, accept, decline
- **NotificationService** — Handles only notification management: create, read, mark-as-read

Open/Closed Principle (OCP)

Classes are designed to be extended without modification:

- **BaseEntity** provides an abstract `validate()` method. Each entity class (User, Task, Project, Invitation, Notification) implements its own validation rules by overriding this method — without modifying the base class.
- **BaseRepository** provides generic CRUD operations. Concrete repositories like TaskRepository add domain-specific methods (`findByAdminAndTitle` , `findByAssignee` , `findByProject`) without altering the base implementation.
- **BaseService** provides `execute()` for error handling and `validateRequired()` for field validation. Concrete services extend these with domain-specific logic without changing the base.

Liskov Substitution Principle (LSP)

Any subtype can seamlessly replace its parent type:

- All entity classes (User, Task, Project, Invitation, Notification) extend BaseEntity and implement IEntity. Any entity can be used wherever IEntity is expected.
- All repository classes extend BaseRepository and implement IRepository. Any repository can substitute where IRepository is expected.
- All service classes extend BaseService and use the same execute() and validateRequired() patterns, ensuring behavioral consistency.

Interface Segregation Principle (ISP)

Interfaces are kept small, focused, and composable:

- **IReadable** — Provides only read operations: `findById()` , `findAll()`
- **IWritable** — Provides only write operations: `create()` , `update()`
- **IDeletable** — Provides only the delete operation: `delete()`
- **IRepository** — Composes all three interfaces plus `findByFilter()` for consumers needing full CRUD

Dependency Inversion Principle (DIP)

High-level modules depend on abstractions rather than concrete implementations:

- API route handlers depend on service interfaces (IAuthService, ITaskService, IUserService, IProjectService) accessed through singleton getInstance() methods.
- Services depend on repository abstractions (IRepository) rather than directly on MongoDB or Mongoose models.

- The edge middleware uses the `jose` library for JWT verification, completely decoupled from the `jsonwebtoken` library used in the service layer.

---

7. Design Patterns

Pattern	Implementation
Repository Pattern	All repositories abstract MongoDB operations behind the generic IRepository interface, providing a clean data access layer
Service Pattern	All business logic is encapsulated in dedicated service classes, preventing logic leakage into API routes
Singleton Pattern	Every repository and service uses a private constructor with a static getInstance() method, ensuring single instances and efficient resource usage
Template Method	BaseEntity defines an abstract validate() method as a contract; each entity subclass implements domain-specific validation rules
Strategy Pattern	The Navbar component dynamically selects navigation link sets based on the current user's role (Admin links, Project Manager links, Developer links, UI/UX Designer links, Member links, or guest links)
Observer Pattern	The task creation API route triggers the NotificationService to create notifications when tasks are assigned to members
Provider Pattern	React Context providers (AuthProvider, ThemeProvider) manage global application state for authentication and theming
Factory Method	BaseEntity.generateId() encapsulates the ID creation logic using timestamp and random suffix

---

8. Database Design

Collections and Schema

Users Collection - Fields: name, email (unique, lowercase), password (hashed), role, avatar, isApproved - The password field is automatically stripped from JSON responses via a toJSON transform - System supports 5 distinct roles for granular access control: **Admin**, **Project Manager**, **Developer**, **UI/UX Designer**, and **Member** - Each role has specific permissions and a tailored dashboard experience

Tasks Collection - Fields: title, description, status (pending/in_progress/completed/archived), priority (low/medium/high/critical), deadline, projectId, assignedTo, assignedBy - A compound unique index on (title + assignedBy) enforces task title uniqueness per admin - Application-level validation also enforces title uniqueness within a project

Projects Collection - Fields: name (unique), description, ownerId, memberIds (array) - The project name is globally unique across the system - The owner is automatically added as the first member

Invitations Collection - Fields: projectId, adminId, memberId, status (pending/accepted/declined) - Application-level duplicate checking prevents multiple pending invitations for the same member

Notifications Collection - Fields: userId, type (task_assigned/task_completed), message, read (boolean), relatedTaskId - Supports unread count queries and batch mark-as-read operations

Entity Relationships

- A User owns multiple Projects (one-to-many via ownerId)
- Users are members of multiple Projects (many-to-many via memberIds array)
- A Project contains multiple Tasks (one-to-many via projectId)
- Tasks are assigned to Users (one-to-many via assignedTo)
- Tasks are assigned by Users (one-to-many via assignedBy)
- Users receive multiple Notifications (one-to-many via userId)
- Notifications optionally reference a Task (via relatedTaskId)
- Users send and receive Invitations for Projects

---

9. Role Definitions

Role	Permissions	Dashboard Access
Admin (Deepanshu)	Full system access, manages all projects, members, and settings	/dashboard
Project Manager (Kanishk Sharma)	Creates and manages projects, assigns tasks, oversees team progress	/dashboard
Developer (Daniel Tayal)	Views and updates development tasks, tracks progress	/member
UI/UX Designer (Dev Tyagi)	Manages design tasks, prototypes, and user experience work	/member
Member (Ritik Ranjan)	Basic team member, views and updates assigned tasks	/member

---

10. Authentication and Security

Authentication Flow

1. User submits login credentials via the login form
2. The POST /api/auth/login route delegates to AuthService.login()
3. AuthService queries UserRepository.findByEmail() to locate the user in MongoDB
4. For member accounts, the isApproved status is checked — unapproved members are rejected
5. bcrypt.compare() verifies the submitted password against the stored hash
6. On success, jwt.sign() generates a JWT token containing userId, email, and role
7. The token is set as an HTTP-only cookie named "taskforge-token" with a 7-day expiry
8. The response includes the user profile (without password) and session details
9. The AuthProvider stores session data in React context and localStorage for client-side access

Security Measures

- **Password Hashing**: All passwords are hashed using bcryptjs with 12 salt rounds before database storage

- **HTTP-Only Cookies**: JWT tokens are stored in HTTP-only cookies to prevent JavaScript access and XSS attacks
- **Edge Middleware**: Next.js middleware intercepts all protected routes and verifies JWT tokens using the jose library
- **Role-Based Access**: The system enforces granular permissions based on 5 distinct roles. Admins have full system access; Project Managers manage projects and tasks; Developers focus on development tasks; UI/UX Designers manage design-related tasks; Members access their assigned tasks only
- **Member Approval**: New member registrations set isApproved to false. Members cannot authenticate until an administrator explicitly approves their account
- **Password Stripping**: The Mongoose UserSchema includes a toJSON transform that removes the password field from all API responses
- **Input Validation**: Services validate required fields before processing, returning structured error messages

---

11. API Design

Authentication Endpoints

- **POST /api/auth/login** — Authenticates user with email and password, returns JWT session
- **POST /api/auth/signup** — Registers new user, hashes password, creates session
- **POST /api/auth/logout** — Clears the authentication cookie

Task Endpoints

- **GET /api/tasks** — Retrieves tasks with optional filters (?assignedTo, ?projectId)
- **POST /api/tasks** — Creates a task with uniqueness validation and automatic notification
- **PATCH /api/tasks/:id** — Updates task fields including status transitions
- **DELETE /api/tasks/:id** — Removes a task

Project Endpoints

- **GET /api/projects** — Retrieves all projects
- **POST /api/projects** — Creates a project with global name uniqueness validation

User Endpoints

- **GET /api/users** — Retrieves all users with passwords excluded
- **POST /api/users** — Creates a user with password hashing
- **PUT /api/users/:id** — Updates user profile or approves pending member
- **DELETE /api/users/:id** — Rejects and deletes a pending member

Invitation Endpoints

- **GET /api/invitations** — Retrieves invitations for a member (?memberId)
- **POST /api/invitations** — Creates an invitation with duplicate prevention
- **PATCH /api/invitations/:id** — Accepts or declines an invitation

Notification Endpoints

- **GET /api/notifications** — Retrieves notifications for a user (?userId)
- **POST /api/notifications** — Creates a new notification
- **PATCH /api/notifications** — Marks all notifications as read for a user (?userId)

---

12. Frontend Design

Page Structure

Home Page (/) — A marketing-style landing page featuring a hero section with parallax scrolling, animated statistics, feature cards with scroll-triggered animations, a "How It Works" section, testimonials with star ratings, and a call-to-action section. Built with Framer Motion for smooth animations.

Login Page (/login) — A clean authentication form with email and password fields, error handling, and automatic role-based redirection upon successful login.

Signup Page (/signup) — A registration form with name, email, password fields, and role selection (Admin, Project Manager, Developer, UI/UX Designer, Member). Includes validation and informational messaging about the approval process.

Admin Dashboard (/dashboard) — A comprehensive management interface with sections for task creation and management, team member management (with approve/reject actions for pending members), project creation and management, and task assignment to team members within specific projects.

Member Page (/member) — A focused interface showing tasks assigned to the current member, notification list with unread indicators, project invitations with accept/decline actions, and task status update buttons for workflow progression.

Component Design

- **Navbar** — Responsive navigation with glassmorphism effect (transparent with backdrop blur), role-based link rendering, theme toggle, user profile display, and mobile hamburger menu with animated drawer
- **TaskCard** — Displays task information with color-coded priority badges, status indicators, deadline countdown with urgency colors (green/amber/red), overdue pulse animation, and action buttons for status progression
- **TeamMemberCard** — Shows member information with approval status and action buttons for admin approval or rejection
- **AnimatedCard** — Wrapper component providing scroll-triggered fade-in animations using Framer Motion's intersection observer
- **ParallaxSection** — Wrapper component applying parallax scroll effects to content sections
- **FormInput** — Reusable input component with icon support and consistent styling
- **StatsCard** — Animated counter display for statistics
- **AuthProvider** — React Context provider managing user authentication state, login/signup/logout functions, and session persistence via localStorage
- **ThemeProvider** — React Context provider managing dark/light theme state with system preference detection and localStorage persistence

---

13. OOP Concepts

Concept	Implementation
Abstraction	BaseEntity, BaseRepository, and BaseService abstract away implementation details behind well-defined interfaces and abstract methods
Encapsulation	Entity classes bundle data and validation logic together. Singleton classes use private constructors to prevent external instantiation

Concept	Implementation
Inheritance	All entities extend BaseEntity. All repositories extend BaseRepository. All services extend BaseService
Polymorphism	Each entity overrides the abstract validate() method with domain-specific rules. Repositories extend base methods with entity-specific queries
Generics	BaseRepository, IRepository, IReadable, IWritable use TypeScript generics for type-safe code reuse across all entity types

---

14. SDLC Methodology

The project follows **Agile** methodology with iterative development:

1. **Planning** — Defined core requirements: authentication, task management, team management, role-based access control
2. **Analysis** — Analyzed user needs, identified entity relationships, defined system specifications
3. **Design** — Created architecture diagrams, class diagrams, ER diagrams, sequence diagrams, and data flow diagrams
4. **Implementation** — Developed the application using Next.js and TypeScript with strict SOLID principles adherence
5. **Testing** — Unit and integration testing configured with Jest and React Testing Library
6. **Deployment** — Deployable to Vercel with MongoDB Atlas for cloud database hosting
7. **Maintenance** — Iterative improvements including member approval workflow, task uniqueness constraints, invitation system, and notification system

---

15. Conclusion

TaskForge demonstrates a professionally architected full-stack web application that strictly adheres to SOLID principles, established design patterns, and clean architecture practices. The Service-Repository pattern ensures clear separation of concerns across all layers of the application. TypeScript interfaces provide strong contracts between modules, enabling safe refactoring and extensibility. The system is designed to be modular and maintainable — new entities, services, or repositories can be added without modifying existing code, following the

Open/Closed Principle. The combination of JWT authentication, role-based edge middleware, bcrypt password hashing, and HTTP-only cookies provides a robust, production-grade security foundation.

The collaborative effort of our 5-member team — Deepanshu (Admin), Dev Tyagi (UI/UX Designer), Kanishk Sharma (Project Manager), Daniel Tayal (Developer), and Ritik Ranjan (Member) — has resulted in a comprehensive, production-ready task management system.